

Unit 4 — Effective Agentic System Design Techniques

System prompts & instruction formatting • State space & environment modelling • Memory architecture & context management • Sequential vs parallel workflows

Contents

- | | |
|---|---|
| 1. CWD implementation: system prompts, instruction formatting, interaction patterns | 8. Integration & interaction patterns |
| 2. Chapter 6 summary & Q&A | 9. Monitoring & adaptation |
| 3. Focused system prompts & instructions | 10. Memory architecture: short-term, long-term, episodic |
| 4. Defining objectives; task specifications | 11. Context management |
| 5. Contextual awareness | 12. Integration with decision-making |
| 6. State space representation | 13. Sequential & parallel processing; workflow optimisation |
| 7. Environment modelling (static / dynamic) | 14. Summary, Q&A |

Part A — Implementing CWD in LLM-Based Systems (end of Chapter 6)

In traditional multi-agent systems, behaviours are explicit code; in LLM-based CWD systems they are controlled by **prompts and interaction patterns**. Three foundations: **system prompts, instruction formatting, interaction patterns**.

A1. System prompts and agent behaviour

System prompts are the **fundamental configuration layer**: unlike regular task prompts, they set an agent's **persistent traits, boundaries and behavioural frameworks** across its operational life cycle. In the travel planner every agent's system prompt must include:

- **Role definition and scope of responsibilities**
- **Constraints and operational boundaries**
- **Communication protocols** with other agents
- **Decision-making frameworks** specific to the role

Example: the **flight booking worker's** prompt has flight search parameters, pricing considerations, airline partnerships; the **Coordinator's** prompt has high-level planning and oversight. In **CrewAI, role** (functional boundaries) + **backstory** (context/expertise, e.g., "former airline revenue management expert...") together form a rich system prompt.

A2. Instruction formatting

Clear, consistent formats give reliable performance and inter-agent communication (instructions are interpreted via NL understanding). Key aspects:

Aspect	Purpose	Format example
Input structuring	Standard formats for task assignments/requests — unambiguous search parameters.	<pre>{ "task_type": "flight_search", "parameters": { "departure": "location", "destination": "location", "dates": "date_range" } }</pre>
Output templates	Consistent, parseable response structures; quick status identification.	<pre>{ "status": "completed/failed", "result": { "options": [...], "recommendations": [...], "constraints": [...] } }</pre>
Communication protocols	Formats for inter-agent messages and status updates: type, sender, recipient, structured content.	<pre>{ "message_type": "update", "sender": "flight_booking_worker", "recipient": "coordinator", "content": { "progress": "in_progress", "completion": "60%", "pending_tasks": [...] } }</pre>

A3. Interaction patterns

- **Message-passing protocols** — structured agent-to-agent formats; clear handoff procedures between processing stages; error handling & recovery.
- **State management** — awareness of current task status; tracking progress through multi-step processes; coordinating parallel activities.
- **Feedback loops** — communicating success/failure; requesting clarification/more information; continuous improvement via interaction history.

A4. Chapter 6 summary & Q&A

CWD adapts early role-based research to LLM agents: well-defined roles, specialised workers under coordinator oversight, delegators managing distribution, effective communication/collaboration patterns, and implementation details (system prompts, instruction formatting) for operational success.

- 1. How does CWD enhance efficiency?** Clear hierarchy — coordinator oversight, delegator distribution, specialised workers; enables **parallel processing** (core travel workers do bookings while analysis workers process data/search opportunities).
- 2. Role & backstory significance?** Role = functional boundaries; backstory = context/expertise shaping decisions; together form the system prompt (e.g., aviation booking specialist + "former airline revenue management expert").
- 3. Core travel workers vs analysis & intelligence workers?** Core: practical bookings/arrangements (flight, hotel, activity, transport). Analysis: data analyst, experience reflector, opportunity searcher — trends, feedback, new opportunities. Core executes immediate tasks; analysis improves decision-making and future performance.
- 4. Communication & collaboration aspects?** Standard protocols (FIPA ACL), coordination mechanisms, negotiation/conflict resolution, knowledge sharing.
- 5. Instruction formatting?** Structured input/output ensures unambiguous communication, clear task specifications, parseable results.

Effective Agentic System Design Techniques

Four sections: focused prompts • state spaces & environment modelling • memory & context • sequential & parallel workflows

Part B1 — Focused System Prompts and Instructions for Agents

Focused instruction directs the agent: it lays down the **objective, constraints and operating context**; clarity and explicitness strongly influence performance.

B1.1 Defining objectives

Well-defined objectives are the foundation of intended functions/behaviours. Travel agent in customer service: "**maximise customer satisfaction by providing personalised travel solutions and resolving queries/issues effectively**", with components:

- **Personalisation** — tailor to preferences, budgets, requirements; gather travel goals, interests, constraints.
- **Problem-solving** — handle queries/issues: booking conflicts, travel advisories, alternatives for disruptions.
- **Effective communication** — clear explanations of solutions/trade-offs; active listening; accurate interpretation of feedback.
- **Continuous improvement** — monitor satisfaction and feedback; refine capabilities; adapt to trends.

Metrics: customer satisfaction ratings, successful resolution of queries/issues, quality of personalised plans. Objectives guide prioritisation within **budget, time, logistical constraints**.

Sample system prompt **Objective:** Act as an expert travel agent to provide personalised travel solutions while maximising customer satisfaction.

Core functions: gather & analyse preferences/constraints/budget; create personalised recommendations & itineraries; resolve travel issues & provide alternatives; communicate clearly and professionally; monitor satisfaction and adapt.

Constraints: stay within budget; prioritise customer safety; follow travel regulations; respect booking deadlines.

Behaviour: clear professional language; show empathy & patience; anticipate needs; transparent pricing; present options with pros/cons; document key requirements and deadlines.

B1.2 Task specifications

Detail **steps, expected outputs and potential challenges** so the agent knows its duties.

Task	Steps	Expected outputs	Potential challenges
Customer interaction & inquiry handling	I. Greet; II. gather info (preferences, budget, dates); III. identify nature of inquiry; IV. provide responses/solutions; V. confirm satisfaction.	Clear concise responses; personalised recommendations/itineraries; booking confirmations/updates.	Ambiguous/incomplete requests; language barriers; conflicting preferences/constraints; emotional or dissatisfied customers.
Flight & accommodation booking	Search flights/accommodation by preferences; compare (price, duration, amenities, ratings); present top choices; confirm and book.	Confirmed flight/hotel bookings; itinerary with travel details; invoices/receipts.	Limited availability; fluctuating prices; changes/cancellations; groups/special accommodations.

Sample task spec — flight inquiry **1. Initial query:** capture departure/arrival, preferred dates & time ranges, special requirements (class, layovers, airlines), confirm budget.

2. Search process: search matching flights; filter by price & preferences; sort by best match (price/duration/stops); check seat availability.

3. Presentation: show top three options; display price, duration, layovers; highlight unique features/restrictions; note cancellation policies.

Outputs: flight comparison summary, booking confirmation, travel advisory notices.

B1.3 Contextual awareness

The backbone of intelligent behaviour: understanding environment, user needs and situational nuances; maintaining **global and local contexts** (international restrictions, seasonal weather, hotel policies, local transport).

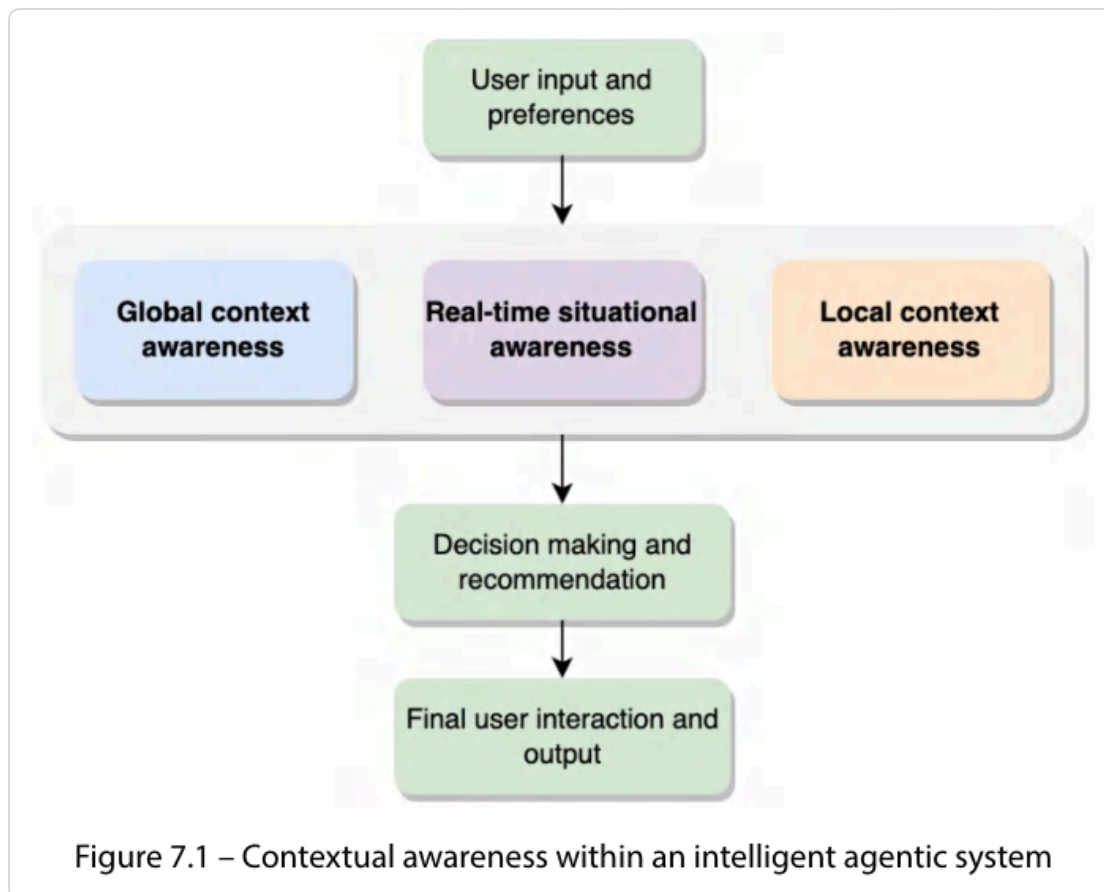


Figure 7.1 — Contextual awareness: user input → global / real-time situational / local context awareness → decision-making & recommendation → final user interaction & output

- **Destination intelligence** — peak seasons, local events, cultural significance (Japan: cherry-blossom season, festivals, regional weather → optimal dates).
- **Dynamic adaptation** — real-time adjustment (flight cancelled by weather → assess alternative routes, impact on later bookings, propose solutions).
- **Cultural competence** — dress codes at religious sites, dietary-friendly restaurants, local customs.

It goes beyond pattern matching: it transforms standard service into curated experiences.

Part B2 — State Spaces and Environment Modelling

B2.1 State space representation

Defines how an agent maintains and updates its understanding of the **current situation, available actions and potential outcomes**; track relevant information while avoiding complexity. Travel agent state space:

Customer profile state	Travel context state	Booking state
Personal preferences & constraints	Flight options & pricing	Reservation status & confirmations
Travel history & feedback	Hotel availability & rates	Payment info & status
Current interaction context	Weather & forecasts	Cancellation policies & deadlines
Budget parameters & flexibility	Travel advisories & restrictions	Itinerary modifications
Special requirements	Seasonal events & peak periods	Connection dependencies

The state space should capture both **static and dynamic elements** efficiently. Example JSON state:

```
{ "booking_id": "BK123456", "status": "confirmed",
  "components": {
    "flights": [{ "status": "confirmed", "departure": "2024-05-15T10:00:00",
                  "cancellation_deadline": "2024-05-01", "dependencies": ["hotel_check_in"] }],
    "hotels": [{ "status": "pending", "check_in": "2024-05-15", "cancellation_policy": "48h_notice" } ] },
  "customer_preferences": { "seat_type": "window", "meal_requirements": "vegetarian",
                             "room_preferences": ["high_floor", "non_smoking"] } }
```

States are "moment-in-time" knowledge about a task; the **environment** is the larger context (tools available, policies/rules, use-case details).

B2.2 Environment modelling

A detailed representation of the world in which the agent operates. Three fundamental questions: **(1)** What systems/services can the agent interact with? **(2)** What rules and constraints govern the interactions? **(3)** What changing conditions must it monitor and respond to?

Travel agent: connections to airline booking systems, hotel reservation platforms, payment processors; business rules; awareness of price changes and availability. A good model enables: informed decisions on current conditions, navigating complex systems, responding to changes, maintaining compliance, optimising within constraints.

Static environment elements (rarely change)	Dynamic environment elements (need real-time monitoring)
Business rules & constraints: booking policies & procedures; payment processing requirements; cancellation/modification rules; service level agreements; regulatory compliance. System interfaces: API endpoints & specifications; database schemas & relationships; authentication mechanisms; error-handling protocols; rate limits & quotas.	Resource availability: real-time inventory; pricing fluctuations; service disruptions; weather; local events. System performance: response times & latency; error rates & failures; resource utilisation; queue lengths & processing times; system health indicators.

Design caution Too many integration points create an overly complex system. Mitigate with **multiple purpose-built agents** that each do one or two tasks well and coordinate (sequential/parallel workflows).

B2.3 Integration and interaction patterns

(1) Event-driven updates

Agent responds dynamically to environment changes; instead of constant **polling** it receives and processes **events** as they occur (airline updates flight status, hotel room becomes unavailable → immediate state updates).

```
class TravelAgentState:
    def update_booking_status(self, event):
        if event.type == "FLIGHT_CHANGE":
            self.check_dependencies()
            self.notify_customer()
        elif event.type == "WEATHER_ALERT":
            self.evaluate_alternatives()
            self.update_recommendations()
```

- **Flight change (10 AM → 2 PM):** system receives `FLIGHT_CHANGE` → method processes it → checks effect on hotel bookings/transfers → notifies customer automatically.
- **Weather alert:** receives `WEATHER_ALERT` → evaluates impact on travel → finds alternative dates/destinations → updates recommendations.

(2) State validation and consistency

Ensures the agent's understanding stays accurate: valid state transitions, maintained dependencies, business rules followed — a **gatekeeper** before state changes are applied.

```
def validate_state_transition(current_state, new_state):
    if not is_valid_transition(current_state, new_state):
        raise InvalidStateTransition("Invalid transition from {current_state} to {new_state}")
    check_state_dependencies(new_state)
    validate_business_rules(new_state)
```

Check	Example
Transition validation	"on hold" → "confirmed": Is payment received? Seats still available? Price still valid?
Dependency checking	Flight + hotel: hotel check-in after flight arrival; transfer availability matches flight time; room type matches number of travellers.
Business rules	International booking: passport information provided; travel insurance offered; cancellation policy acknowledged.

If any check fails, the state change is blocked and an error raised → booking integrity maintained.

B2.4 Monitoring and adaptation

- **Monitoring metrics:** latency of state updates; accuracy/precision of predictions and decisions; resource-utilisation patterns; error rates & recovery times; **customer-satisfaction indicators**.
- **Adaptation strategies:** dynamic resource allocation for varying workloads; continuously refine/update models from new data; adjust rules & optimisation parameters (seasonal patterns, preference shifts); performance tuning & scaling; incorporate user feedback.

Success depends on representing state and environment and adapting: consider static + dynamic elements, robust integration patterns, effective monitoring — keeping the agent reliable, efficient and responsive as the environment evolves.

Part B3 — Agent Memory Architecture and Context Management

Memory enables coherent interactions and informed decisions from past experience and current context. Three memory types: **short-term, long-term, episodic**.

Memory type	Purpose	Travel agent example
Short-term (working) memory	Immediate cognitive workspace; temporarily holds info for the current interaction/task; conversation context, multi-step processes, active sessions; cleared when session ends or info becomes irrelevant (efficient resource use).	Current search criteria, recently viewed options, temporary preferences during the session, booking-process state.
Long-term memory (knowledge base)	Persistent repository across interactions/sessions; accumulated knowledge, learned patterns, customer relationships; consistency and personalisation without repeating preferences.	Customer preferences over multiple bookings, past travel records, destination knowledge, seasonal patterns, provider relationships.
Episodic memory (interaction history)	Stores specific interactions, events and outcomes as discrete episodes with temporal sequence & context; learn from past experience; pattern recognition on behaviour and outcomes; common in conversational interfaces/chatbots.	Identify successful booking patterns, common customer journeys; avoid repeat issues (e.g., problem layover times at certain airports).

Implementation sketches

```
# Short-term memory
class WorkingMemory:
    def __init__(self):
        self.current_interaction = {'customer_id': None, 'session_start': None,
                                    'current_query': None, 'active_searches': [], 'temporary_preferences': {}}
    def update_context(self, new_information): # update current interaction context
        self.current_interaction.update(new_information)
    def clear_session(self): # reset temporary session data
        self.__init__()
```

```
# Long-term memory
class CustomerMemory:
    def __init__(self):
        self.profiles = {'preferences': {}, 'travel_history': [], 'feedback_history': [],
                        'special_requirements': {}, 'loyalty_status': None}
    def update_profile(self, customer_id, new_data): # merge new info with existing profile
        self.profiles[customer_id] = {**self.profiles.get(customer_id, {}), **new_data}

class TravelKnowledge:
    def __init__(self):
        self.destination_info = {}; self.seasonal_patterns = {}
        self.service_providers = {}; self.travel_regulations = {}
    def update_knowledge(self, category, key, value):
        getattr(self, category)[key] = value
```

```
# Episodic memory
class EpisodicMemory:
    def __init__(self): self.interaction_history = []
    def record_interaction(self, interaction_data):
        interaction_data['timestamp'] = datetime.now()
        self.interaction_history.append(interaction_data)
    def retrieve_relevant_episodes(self, context): # similar past interactions
        return [e for e in self.interaction_history if self._is_relevant(e, context)]
```

B3.1 Context management

ScenarioMulti-city business trip to Tokyo & Singapore: corporate policy limits flights to \$2,000, preference for morning flights due to meetings, hotels within walking distance of offices — the agent must track and update all of this while moving between flight searches, hotel availability and meeting schedules.

Context hierarchy

Level	Contents
Global context	System-wide settings & constraints; current operational status; global travel alerts and advisories.
Session context	Current customer-interaction state; active searches/queries; temporary preferences & constraints.
Task context	Specific booking details; current step in multi-step processes; related bookings & dependencies.

Context switching — smooth transition between contexts while preserving coherence:

- **Context preservation:** save current state before switching; maintain history of context changes; ensure no critical information is lost.
- **Context restoration:** retrieve previous contexts; rebuild operational environment; re-establish relevant connections and states.
- **Context merging:** combine information from multiple contexts; resolve conflicts; maintain consistency across changes.

B3.2 Integration with decision-making

1. **Information retrieval** — gather/synthesise from memory: customer history & preferences; similar past cases & outcomes; combine current context with history; filter & prioritise.
2. **Pattern recognition** — analyse historical interaction patterns; identify successful booking patterns; recognise potential issues from past experience; detect seasonal trends & preferences.
3. **Decision optimisation** — weighted evaluation of options; multiple constraints; balance customer preferences vs system requirements; risk assessment and mitigation.

Part B4 — Sequential and Parallel Processing in Agentic Workflows

B4.1 Sequential processing

Tasks handled in a **defined order**, each step depending on the completion of previous ones. Needed for: (1) **Flight & hotel coordination** — confirm flight availability before booking hotels; transfers aligned with arrival times; validate visa requirements before finalising; (2) **Payment processing** — verify funds → process payment → confirm bookings → send confirmation documents.

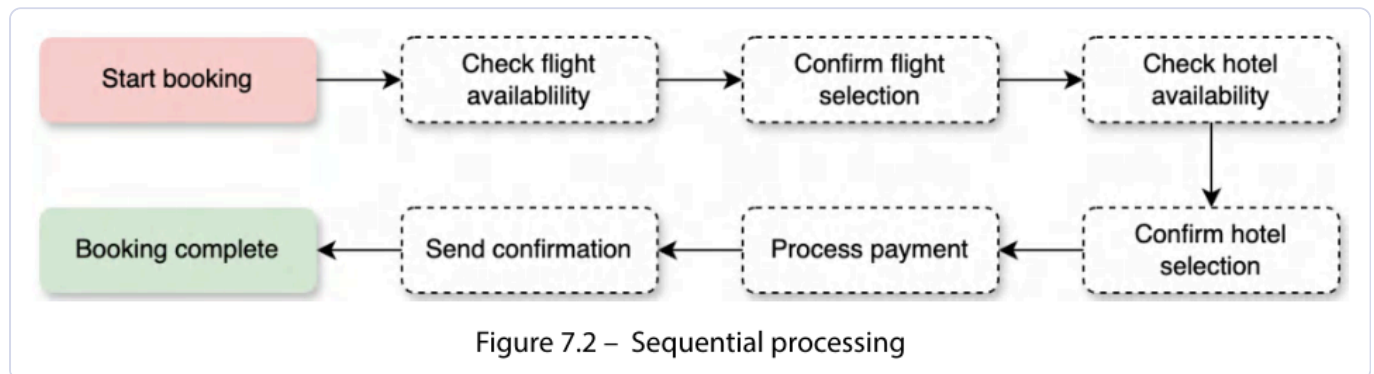


Figure 7.2 — Sequential processing: Start booking → Check flight availability → Confirm flight selection → Check hotel availability → Confirm hotel selection → Process payment → Send confirmation → Booking complete

Drawback: inefficient when many independent operations could run simultaneously.

B4.2 Parallel processing

Multiple independent tasks handled **simultaneously** → better efficiency and response times. (1) **Concurrent searches** — multiple airline systems, different hotel chains, weather forecasts & travel advisories; (2) **Background processing** — update customer profiles, process feedback & reviews, monitor price changes, update travel advisories.

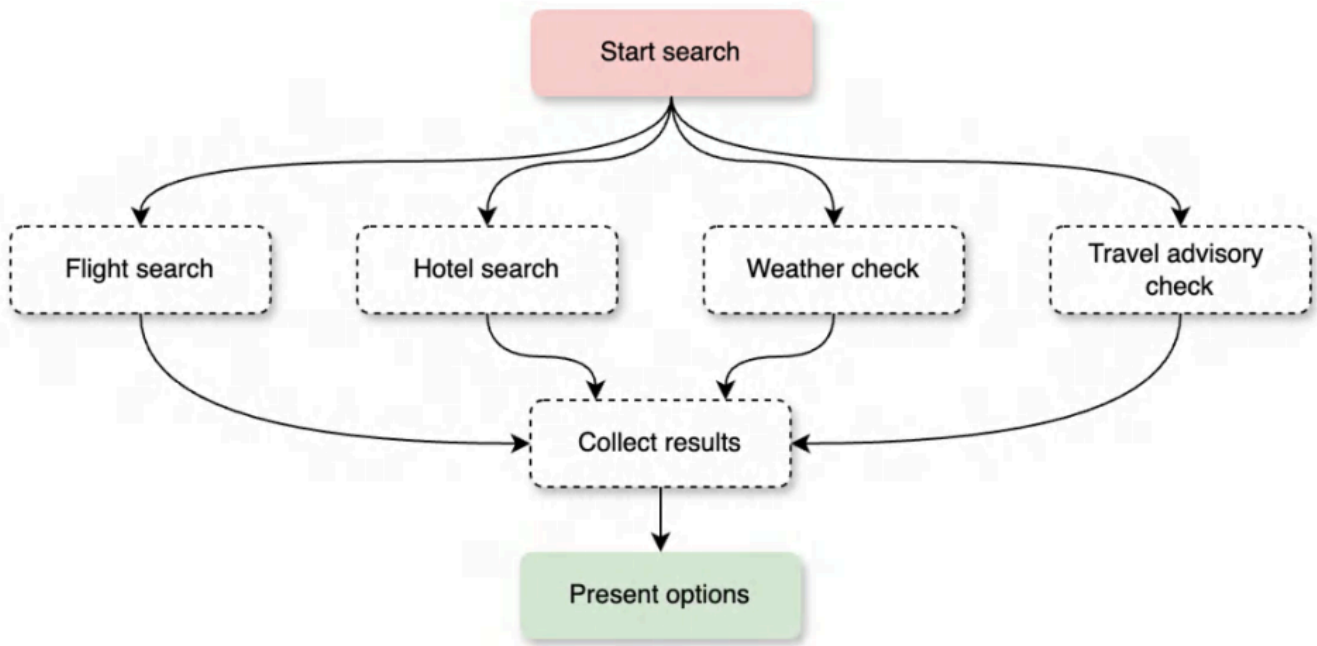


Figure 7.3 – Parallel processing

Figure 7.3 — Parallel processing: Start search → (Flight search | Hotel search | Weather check | Travel advisory check) → Collect results → Present options

Sequential	Parallel
Use when tasks have dependencies (payment → confirmation)	Use for independent tasks (multi-vendor searches)

An agent must dynamically switch modes depending on task requirements, system load and time constraints → **workflow optimisation**.

B4.3 Workflow optimisation

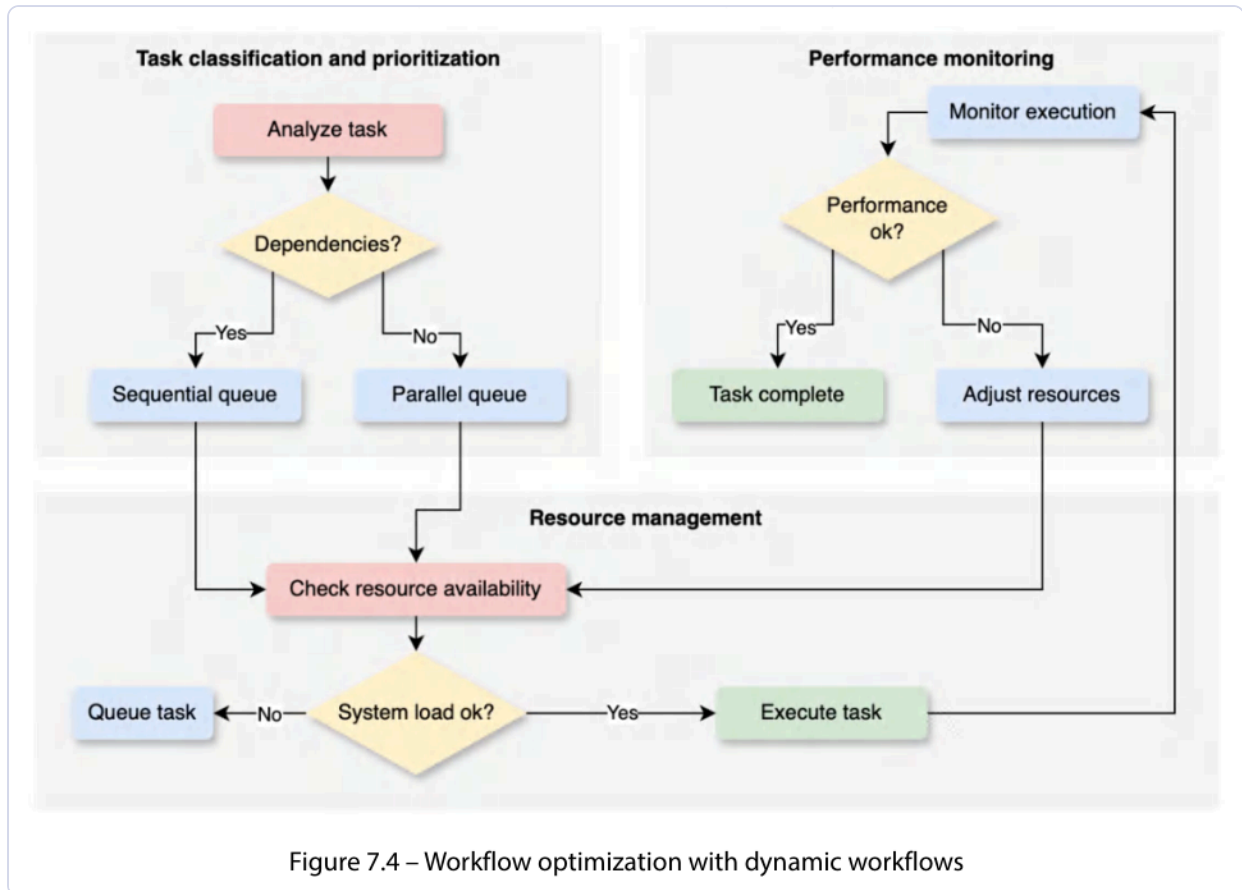


Figure 7.4 – Workflow optimization with dynamic workflows

Figure 7.4 — Workflow optimisation with dynamic workflows (task classification & prioritisation, resource management, performance monitoring)

1. Task classification & prioritisation

- *Dependency analysis*: critical-path tasks that must be sequential; map dependencies between booking components; data-flow requirements; temporal constraints & deadlines.
- *Priority assignment*: urgency/importance; customer SLAs & expectations; impact on overall booking; resource requirements.

2. Resource management

- *System resources*: monitor CPU/memory; balance load across components; **throttling**; optimise DB connections & caches.
- *External services*: track API **rate limits/quotas**; manage concurrent requests; **retry strategies** for failures; maintain provider priorities.

3. Dynamic workflow adjustment

- *Load balancing*: adjust parallel execution to load; redistribute during peaks; manage queue depths & processing rates; implement **backpressure**.
- *Performance monitoring*: task completion times & success rates; identify bottlenecks; measure throughput & latency; monitor resource utilisation.

Goal: workflows that are efficient, **resilient and adaptable**.

Flow in Fig 7.4: Analyse task → Dependencies? (Yes: sequential queue / No: parallel queue) → Check resource availability → System load OK? (No: queue task / Yes: execute task) → Monitor execution → Performance OK? (Yes: task complete / No: adjust resources)

Chapter 7 Summary and Q&A

- **Focused system prompts** (objective, core functions, constraints, behaviour), **task specifications** (steps, outputs, challenges), **contextual awareness** (destination intelligence, dynamic adaptation, cultural competence).
- **State space** = moment-in-time view (customer profile, travel context, booking state); **environment model** = systems, rules, changing conditions (static vs dynamic).
- Integration patterns: **event-driven updates** and **state validation & consistency**; monitoring + adaptation.
- **Memory**: short-term (working), long-term (knowledge base), episodic (interaction history); **context hierarchy** (global/session/task), switching (preservation, restoration, merging).
- **Workflows**: sequential for dependent tasks, parallel for independent tasks; optimisation by classification, resource management, dynamic adjustment. (Next chapter: building trust in generative-AI systems.)

Q1. Three memory types & importance? Short-term (immediate context, active search parameters, session state), long-term (persistent profiles, regulations, destination info), episodic (interaction episodes enabling learning & pattern recognition) — maintain context, learn from experience, decide using current + historical info.

Q2. Sequential vs parallel; when in travel booking? Sequential for dependent steps (flight availability before hotel; payment → confirmation); parallel for independent tasks (airline searches, hotel availability, weather forecasts at the same time).

Q3. Context management & levels? Maintains awareness of current situation + history: **global, session, task** contexts.

Q4. Environment model vs state space? Environment model = external systems, static rules (policies, interfaces) and dynamic elements (availability, pricing); state space = current situation, available actions, outcomes — environment gives the broader context in which the state space exists.

Q5. Factors for sequential vs parallel? Task dependencies, resource availability, time constraints (SLAs), system load, performance requirements (throughput/latency).

Likely exam questions 1) Explain system prompts, instruction formatting (input/output/communication JSON) and interaction patterns. 2) Differentiate state space and environment modelling; static vs dynamic elements. 3) Explain event-driven updates and state validation with code. 4) Describe the three memory types with classes and the context hierarchy. 5) Compare sequential and parallel workflows and explain workflow optimisation.

Sources: Unit-4 (end of Ch.6 CWD approach + Ch.7 Effective Agentic System Design Techniques) — summarised for RVCE study use.